Autómata Celular de Tráfico con OpenMP: análisis serial, paralelo y optimización de memoria

Reto 2 — Computación de Alto Rendimiento

Juan Camilo Galvis Daniel Toro Soto

Mayo 2026

Resumen

El presente documento describe la implementación y análisis de rendimiento de un autómata celular de tráfico unidimensional (Rule 184 / modelo de Nagel-Schreckenberg simplificado) bajo tres estrategias de cómputo en lenguaje C: una versión serial de referencia, una versión paralela con OpenMP que combina paralelismo en el bucle de densidades (*embarrassingly parallel*, Opción 2) y paralelismo en el bucle de celdas (Opción 1), y una versión que añade optimizaciones de memoria (`uint8_t` en lugar de `int` y un pool de arreglos pre-allocado). Las pruebas se realizaron en un Apple Mac Mini (2024) con procesador M4 de 10 núcleos y 16 GB de memoria unificada, barriendo longitudes de carretera desde $N = 10\,000$ hasta $N = 300\,000$ con 20 niveles de densidad. La versión paralela logra un speedup de 3,4×–4,1× sobre la serial. La versión con optimizaciones de memoria, aunque corrige el exceso de asignaciones dinámicas y reduce la huella de caché, resulta marginalmente más lenta (≈5 %) que la paralela pura, evidenciando que en este algoritmo el cuello de botella es la aritmética de índices y no el ancho de banda de memoria.

Índice

1. Introducción **3**

2. Marco Conceptual **3**
2.1. High Performance Computing (HPC) . . . 3
2.2. Autómata Celular de Tráfico (Rule 184) . . . 3
2.3. OpenMP . . . 3
2.4. Optimización de memoria . . . 4
2.5. Speedup y Ley de Amdahl . . . 4

3. Marco Contextual **4**
3.1. Características de la máquina . . . 4
3.2. Implementaciones desarrolladas . . . 5
3.3. Sistema de compilación (Makefile) . . . 5
3.4. Sistema de benchmarking con checkpoints . . . 6

4. Resultados **6**
4.1. Secuencial . . . 6
4.2. Paralelo (OpenMP) . . . 6
4.3. Memoria + OpenMP . . . 7
4.4. Tabla resumen de speedups . . . 7
4.5. Gráficas comparativas . . . 8
4.5.1. Tiempos de ejecución . . . 8
4.5.2. Speedup relativo al secuencial . . . 9

5. Análisis de resultados **9**
5.1. Impacto del paralelismo (Opción 2 + Opción 1) . . . 9
5.2. Impacto de las optimizaciones de memoria . . . 9
5.3. Escalado con N y tendencia asintótica . . . 10
5.4. Comparación entre las tres implementaciones . . . 10

6. Conclusiones **10**

7. Bibliografía **12**

1. Introducción

Los autómatas celulares constituyen modelos discretos de gran valor en la simulación de sistemas complejos. Uno de sus usos más estudiados es la modelización del flujo de tráfico vehicular: la *Rule 184* del autómata celular elemental captura cualitativamente los fenómenos de flujo libre, congestión y la transición entre ambos regímenes [3]. A pesar de la sencillez de sus reglas locales, la simulación de carreteras largas con múltiples niveles de densidad representa un desafío computacional significativo.

En este reto se parte de una implementación serial correcta y se aplica OpenMP para explorar dos estrategias de paralelización independientes: (1) paralelizar el bucle interno de celdas, que proporciona una reducción nativa de la variable de velocidad, y (2) paralelizar el bucle externo de densidades, que es completamente independiente entre iteraciones (*embarrassingly parallel*). Adicionalmente, se investiga si el uso de arreglos de menor tamaño (`uint8_t`) y un esquema de allocación pre-calculada mejoran el rendimiento respecto a la versión paralela base.

El objetivo es cuantificar el speedup real de cada estrategia, comprender las causas de los resultados observados y extraer conclusiones sobre qué técnicas de HPC son más efectivas para este tipo de problema en la arquitectura Apple M4.

2. Marco Conceptual

2.1. High Performance Computing (HPC)

La Computación de Alto Rendimiento agrupa técnicas y arquitecturas orientadas a resolver problemas con elevados requisitos computacionales en el menor tiempo posible. Como señala López (2017), "hacemos referencia a un campo de la computación actual que da solución a problemas tecnológicos muy complejos y que involucran un gran volumen de cálculos o de coste computacional". En este reto, el producto cartesiano entre longitudes de carretera y niveles de densidad genera un trabajo total del orden de $\mathcal{O}(N^2)$ operaciones de celda, justificando el uso de paralelismo explícito y optimización de accesos a memoria.

2.2. Autómata Celular de Tráfico (Rule 184)

Un autómata celular unidimensional consiste en N celdas dispuestas en un anillo circular. Cada celda puede estar ocupada (1) o vacía (0). En cada paso discreto de tiempo, el estado siguiente de cada celda se calcula simultáneamente a partir de su vecindad local [3]:

- **Celda vacía** ($s_i = 0$): adopta el estado del vecino izquierdo ($s_i^{(t+1)} = s_{i-1}^{(t)}$). Un coche llega si venía de la izquierda.
- **Celda ocupada** ($s_i = 1$): avanza si la celda adelante está libre ($s_i^{(t+1)} = s_{i+1}^{(t)}$); en caso contrario permanece.

Esta regla corresponde a la Rule 184 del autómata elemental de Wolfram y captura la dinámica del flujo vehicular unidireccional. La **velocidad media asintótica** (fracción de coches que se mueven por paso) es la métrica del modelo y describe el *diagrama fundamental de tráfico*: para densidades bajas ($\rho < 0{,}5$) todos los coches se mueven libremente; a $\rho = 0{,}5$ el flujo es máximo; para $\rho > 0{,}5$ el sistema entra en congestión y la velocidad decae [4].

En este estudio la velocidad media se calcula pero no se imprime; el objetivo HPC es minimizar el **tiempo de cómputo total** para barrer los 21 puntos de densidad de $\rho = 0$ a $\rho = 1$.

2.3. OpenMP

OpenMP es una API de programación paralela de memoria compartida basada en directivas de preprocesador. Permite paralelizar bucles con mínimos cambios al código fuente [1]:

```
#pragma omp parallel for schedule(static) reduction(+:total)
for (int i = 0; i < N; i++) {
    total += arr[i];
}
```

Para el autómata celular se identificaron dos opciones de paralelización:

Opción 1 — Bucle de celdas (*inner loop*): Cada hilo procesa un bloque contiguo de N/T celdas. Requiere una `reduction` para acumular `moved_this_step`. Se aplica a los bucles de calentamiento y medición dentro de `simulate()`.

Opción 2 — Bucle de densidades (*outer loop*): Cada hilo simula un punto de densidad completo de forma independiente. Sin sincronización ni variables compartidas: es *embarrassingly parallel*. Se usa `schedule(dynamic)` porque distintas densidades pueden converger en diferente número de pasos. Cada hilo dispone de sus propios arreglos de carretera.

La implementación final combina ambas opciones con paralelismo anidado: `omp_set_nested(1)` habilita dos niveles; el nivel externo (densidades) usa todos los núcleos disponibles; el nivel interno (celdas) usa un único hilo por hilo externo, evitando sobresubscripción.

2.4. Optimización de memoria

El documento `docs/MemoryExplanation.md` identifica dos vectores de mejora:

1. **Reemplazar `int` por `uint8_t`**: cada celda solo almacena 0 ó 1, por lo que los 4 bytes de un `int` desperdician el 75 % del espacio. Con `uint8_t` la huella de los arreglos se reduce a la cuarta parte, permitiendo que carreteras de $N = 300\,000$ celdas (300 KB por arreglo con `uint8_t` vs. 1.2 MB con `int`) quepan mejor en caché.

2. **Pool de arreglos pre-allocado**: en la versión paralela base, cada hilo invoca `malloc` y `free` en cada punto de densidad dentro de la región paralela. El asignador de memoria usa un *mutex* interno, convirtiendo estas asignaciones en una contención serializada. El pool pre-alloca $T \times N$ bytes para cada arreglo *antes* de la región paralela y cada hilo accede a su porción mediante aritmética de punteros.

2.5. Speedup y Ley de Amdahl

El speedup mide la ganancia de rendimiento de la versión paralela respecto a la serial:

$$S_p = \frac{T^*(n)}{T_p(n)}$$

donde $T^*(n)$ es el tiempo secuencial y $T_p(n)$ el tiempo con p hilos. La Ley de Amdahl [2] establece que la fracción serial f del algoritmo limita el speedup:

$$S_p \leq \frac{1}{f + (1-f)/p}$$

Para la Opción 2 la fracción serial es mínima (solo inicialización de semillas y la medición del tiempo final), lo que explica los speedups cercanos a 4× con todos los P-cores del M4.

3. Marco Contextual

3.1. Características de la máquina

Las pruebas se realizaron íntegramente en el siguiente hardware:

- **Equipo:** Apple Mac Mini (2024)
- **Procesador:** Apple M4 — 10 núcleos (4 de rendimiento + 6 de eficiencia)
- **GPU integrada:** 10 núcleos
- **Memoria:** 16 GB RAM unificada LPDDR5X (≈120 GB/s de ancho de banda) [6]
- **Almacenamiento:** 512 GB SSD NVMe
- **Sistema operativo:** macOS Sequoia 15.x (Darwin 25.x)
- **Compilador:** Apple Clang (LLVM), GCC instalado vía Homebrew

- **libomp:** instalada vía Homebrew (requerida para OpenMP en Apple Silicon)

La arquitectura M4 tiene dos particularidades importantes: (1) la **memoria unificada** elimina la latencia de transferencia CPU↔RAM, ofreciendo un ancho de banda ≈3× mayor que una arquitectura x86 típica; (2) los **núcleos heterogéneos** (4 P-cores de alto rendimiento y 6 E-cores de eficiencia) implican que el scheduler de macOS puede asignar hilos a núcleos de distintas capacidades, afectando el speedup efectivo cuando se usan más de 4 hilos.

3.2. Implementaciones desarrolladas

Las tres implementaciones en C comparten la misma lógica base: inicialización aleatoria de la carretera con `rand_r` (libre de estado global), fase de calentamiento de $N/10$ pasos para alcanzar el estado estacionario y fase de medición de $N/10$ pasos. El tiempo total de ejecución (incluyendo el barrido completo de densidades) se mide con `clock_gettime(CLOCK_MONOTONIC)` e imprime en stdout como `printf("%.6f,", elapsed)`.

`secuential` Implementación serial pura. Arreglos de `int`. Un único hilo procesa secuencialmente todos los 21 puntos de densidad. Sirve de referencia para el cálculo del speedup.

`parallel` Combina Opción 1 (bucle de celdas paralelo dentro de `simulate()`) y Opción 2 (bucle de densidades paralelo en `main()`). Arreglos de `int`. Cada hilo alloca y libera sus propios arreglos por punto de densidad (`malloc`/`free` dentro de la región paralela). Usa `schedule(dynamic)` en el nivel externo.

`memory` Extiende `parallel` con dos optimizaciones de memoria: arreglos de `uint8_t` (4× menos memoria) y un pool pre-allocado de $T \times N$ celdas por arreglo, eliminando toda asignación dinámica dentro de la región paralela.

3.3. Sistema de compilación (Makefile)

El `Makefile` detecta el sistema operativo para seleccionar las banderas correctas de OpenMP:

```
CC = gcc
CFLAGS = -O2
UNAME_S := $(shell uname -s)
ifeq ($(UNAME_S),Darwin)
    LIBOMP_PREFIX := $(shell brew --prefix libomp 2>/dev/null)
    OMP_FLAGS = -Xpreprocessor -fopenmp \
                -I$(LIBOMP_PREFIX)/include \
                -L$(LIBOMP_PREFIX)/lib -lomp
else
    OMP_FLAGS = -fopenmp
endif

all: output/secuential output/parallel output/memory

output/secuential: src/SecuentialCellularAutomaton.c
    $(CC) $(CFLAGS) -o $@ $< -lm

output/parallel: src/ParallelCellularAutomaton.c
    $(CC) $(CFLAGS) -o $@ $< -lm $(OMP_FLAGS)

output/memory: src/MemoryCellularAutomaton.c
    $(CC) $(CFLAGS) -o $@ $< -lm $(OMP_FLAGS)
```

El condicional es necesario porque en macOS la cadena de herramientas de Apple Clang no incluye OpenMP de forma nativa: la librería `libomp` debe instalarse por separado (`brew install libomp`) y requiere las banderas `-Xpreprocessor -fopenmp` más las rutas explícitas de inclusión y enlazado. En Linux basta con `-fopenmp`, bandera estándar soportada por GCC y por Clang. El binario `secuential` compila *sin* `OMP_FLAGS` porque no usa la API de OpenMP; los otros dos sí la requieren. Todos se compilan con `-O2` para garantizar una comparación justa entre versiones.

3.4. Sistema de benchmarking con checkpoints

Dado que barrer 13 tamaños × 2 repeticiones para las tres implementaciones puede requerir horas de cómputo, el script `scripts/RunAll.sh` implementa un sistema de **reanudación automática** basado en checkpoints:

Flujo de ejecución:

1. Al iniciar, el script busca en `stats/<hostname>/` si existe un directorio de corrida *sin* el sufijo `.done`. Si existe, lo reanuda; si no, crea uno nuevo con marca de tiempo `YYYYMMDD_HHMMSS`.
2. Cada medición exitosa registra una clave en el archivo oculto `.checkpoint` con el formato `<tipo>,<N>,run<j>` (e.g. `parallel,100000,run1`).
3. La función `run_safe()` consulta `.checkpoint` antes de cada ejecución. Si la clave ya existe, imprime `[SKIP]` y continúa. Si el programa falla con código de salida distinto de cero, emite `[WARN]` y prosigue sin abortar toda la corrida.
4. Al terminar con éxito, el directorio recibe el sufijo `.done`, impidiendo que futuras ejecuciones lo traten como corrida incompleta.

La estructura de salida resultante es:

```
stats/<hostname>/<timestamp>.done/
  .checkpoint        # claves de ejecuciones completadas
  secuential.csv     # tiempos de todas las corridas (,separados)
  parallel.csv
  memory.csv
```

Los tamaños evaluados son: $N \in \{10\,000;\ 25\,000;\ 50\,000;\ 75\,000;\ 100\,000;\ 125\,000;\ 150\,000;\ 175\,000;\ 200\,000;\ 225\,000;\ 2$
con **2 repeticiones** por configuración y `density_steps` $= 20$.

4. Resultados

A continuación se presentan los tiempos de ejecución en segundos para cada una de las tres implementaciones (2 repeticiones) y los speedups derivados. Los datos provienen de la corrida `20260503_230658.done` en el Mac Mini M4.

4.1. Secuencial

Secuencial (tiempo en segundos)													
N	**10k**	**25k**	**50k**	**75k**	**100k**	**125k**	**150k**	**175k**	**200k**	**225k**	**250k**	**275k**	**300k**
Run 1	0.2125	1.1885	4.9630	11.8873	22.6907	38.8417	59.2329	85.0378	115.3182	150.8857	191.7560	236.8075	287.8714
Run 2	0.1887	1.1919	4.9496	11.8519	22.9008	38.5496	58.5546	83.8931	114.1768	149.9194	190.0832	236.2089	287.6727
Prom.	**0.2006**	**1.1902**	**4.9563**	**11.8696**	**22.7958**	**38.6957**	**58.8937**	**84.4655**	**114.7475**	**150.4025**	**190.9196**	**236.5082**	**287.7721**

Cuadro 1: Tiempos de ejecución de la versión secuencial en Mac Mini M4.

4.2. Paralelo (OpenMP)

Paralelo con OpenMP (tiempo en segundos)													
N	**10k**	**25k**	**50k**	**75k**	**100k**	**125k**	**150k**	**175k**	**200k**	**225k**	**250k**	**275k**	**300k**
Run 1	0.0664	0.3176	1.2829	2.9442	5.8332	10.0089	15.2858	21.8329	29.5281	37.6471	46.9762	58.4963	70.0379
Run 2	0.0513	0.3250	1.2800	2.9104	5.7570	10.0077	15.8700	23.3761	29.1194	37.7589	47.2736	58.5032	69.6213
Prom.	**0.0589**	**0.3213**	**1.2814**	**2.9273**	**5.7951**	**10.0083**	**15.5779**	**22.6045**	**29.3238**	**37.7030**	**47.1249**	**58.4998**	**69.8296**
Speedup	**3.41**	**3.70**	**3.87**	**4.05**	**3.93**	**3.87**	**3.78**	**3.74**	**3.91**	**3.99**	**4.05**	**4.04**	**4.12**

Cuadro 2: Tiempos y speedup de la versión paralela (OpenMP) en Mac Mini M4.

4.3. Memoria + OpenMP

Memoria + OpenMP (tiempo en segundos)													
N	**10k**	**25k**	**50k**	**75k**	**100k**	**125k**	**150k**	**175k**	**200k**	**225k**	**250k**	**275k**	**300k**
Run 1	0.0729	0.3427	1.4008	3.2327	6.1833	10.6175	15.9524	22.4107	30.1444	39.1709	48.9081	60.9032	72.8603
Run 2	0.0618	0.3591	1.4040	3.2123	6.1553	10.2836	15.9238	22.8334	30.3222	38.8127	49.4835	61.0987	73.6244
Prom.	**0.0673**	**0.3509**	**1.4024**	**3.2225**	**6.1693**	**10.4505**	**15.9381**	**22.6221**	**30.2333**	**38.9918**	**49.1958**	**61.0010**	**73.2424**
Speedup	**2.98**	**3.39**	**3.53**	**3.68**	**3.70**	**3.70**	**3.70**	**3.73**	**3.80**	**3.86**	**3.88**	**3.88**	**3.93**

Cuadro 3: Tiempos y speedup de la versión Memoria+OpenMP (`uint8_t` + pool pre-allocado) en Mac Mini M4.

4.4. Tabla resumen de speedups

N	Seq (s)	Paralelo (s)	Sp Paralelo	Sp Memoria
10 000	0.2006	0.0589	3,41×	2,98×
25 000	1.1902	0.3213	3,70×	3,39×
50 000	4.9563	1.2814	3,87×	3,53×
75 000	11.8696	2.9273	4,05×	3,68×
100 000	22.7958	5.7951	3,93×	3,70×
125 000	38.6957	10.0083	3,87×	3,70×
150 000	58.8937	15.5779	3,78×	3,70×
175 000	84.4655	22.6045	3,74×	3,73×
200 000	114.7475	29.3238	3,91×	3,80×
225 000	150.4025	37.7030	3,99×	3,86×
250 000	190.9196	47.1249	4,05×	3,88×
275 000	236.5082	58.4998	4,04×	3,88×
300 000	287.7721	69.8296	4,12×	3,93×

Cuadro 4: Resumen de tiempos promedio y speedups para las tres implementaciones.

4.5. Gráficas comparativas

4.5.1. Tiempos de ejecución

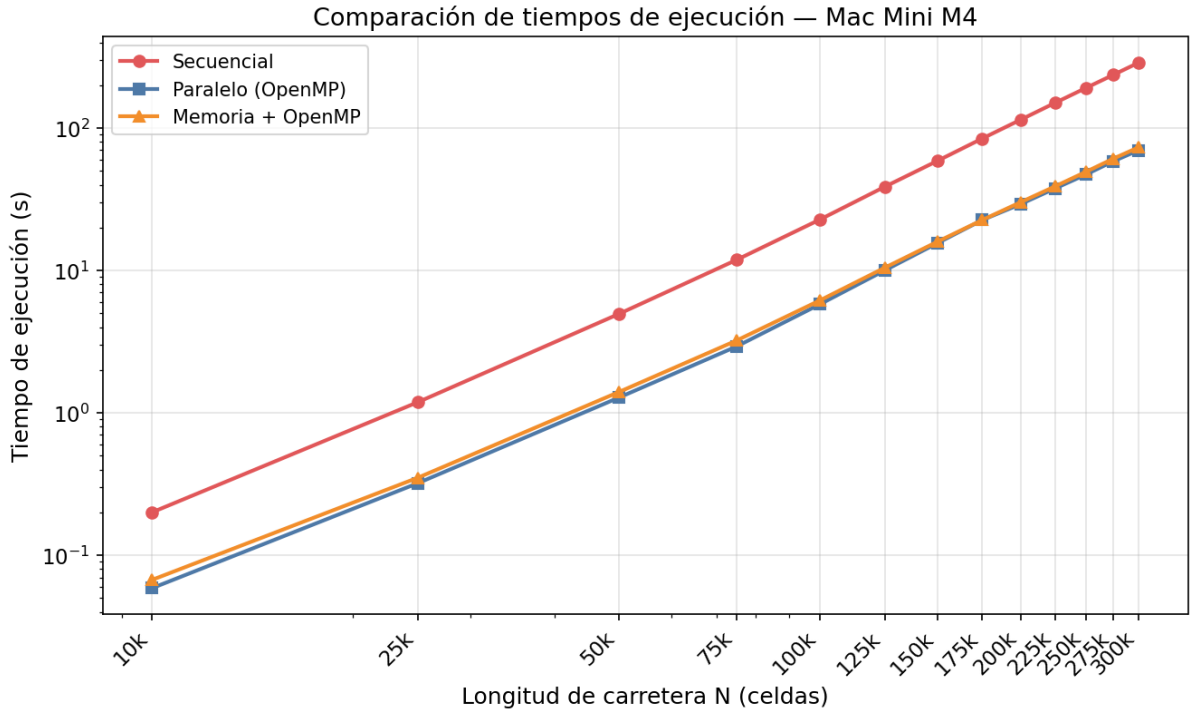

Figura 1: Tiempos de ejecución (escala log-log) en función de la longitud de carretera N para las tres implementaciones en el Mac Mini M4. La versión paralela (azul) y la versión Memoria+OpenMP (naranja) se agrupan en la parte inferior, ambas 3,4×–4,1× por debajo de la secuencial (rojo).

4.5.2. Speedup relativo al secuencial

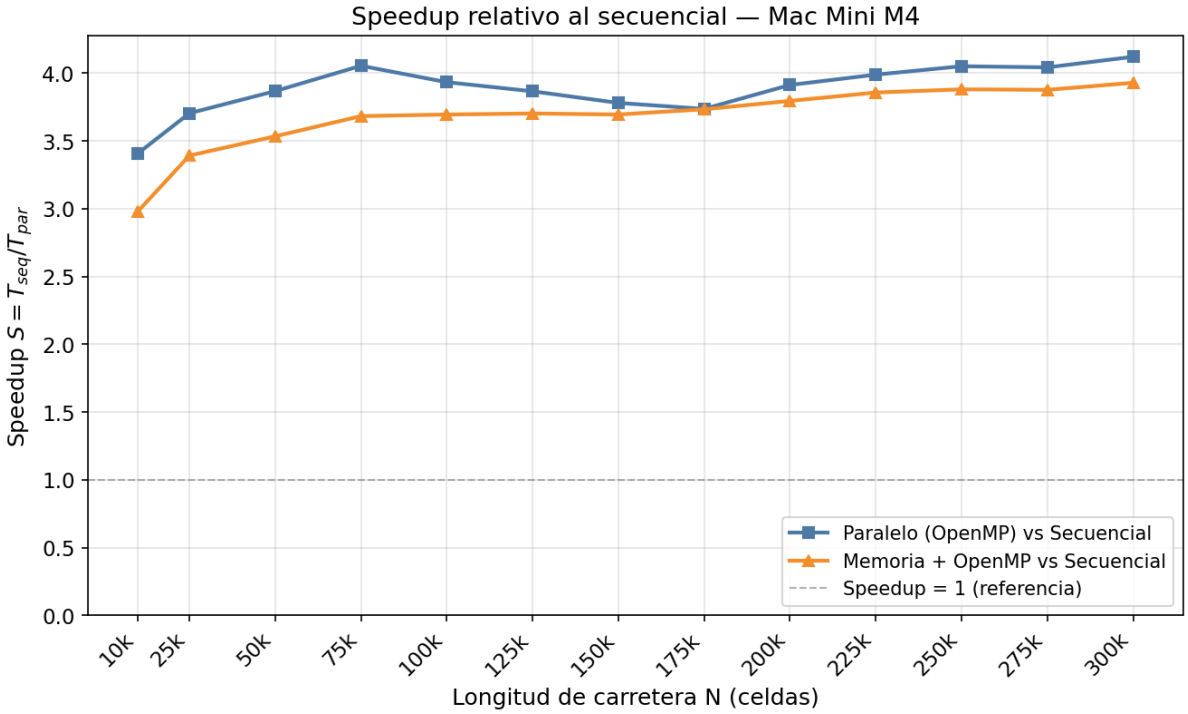

Figura 2: Speedup $S = T_{\text{seq}}/T_{\text{par}}$ en función de N. La versión paralela (azul) mantiene un speedup de 3,4× a 4,1×, creciendo suavemente con N. La versión Memoria+OpenMP (naranja) parte de 2,98× para N pequeño y converge hacia ≈ 3,9× para N grandes.

5. Análisis de resultados

5.1. Impacto del paralelismo (Opción 2 + Opción 1)

La versión paralela logra un speedup sostenido de 3,4×–4,1× con todos los núcleos del M4. Este rango es coherente con la capacidad de los 4 P-cores: al ejecutar con `omp_get_max_threads()` los 10 núcleos participan, pero los 6 E-cores procesan significativamente más lento que los 4 P-cores. El scheduler de macOS distribuye los 21 hilos de densidad entre todos los núcleos; los hilos asignados a E-cores tardan más, creando un desbalance que limita el speedup real a ≈ 4× en lugar del ideal de 10×.

El `schedule(dynamic)` en el bucle externo mitiga parcialmente este desbalance: los hilos en P-cores que terminan antes recogen nuevas densidades, reduciendo el tiempo de espera en la barrera implícita al final del bucle paralelo. No obstante, con solo 21 iteraciones disponibles y 10 hilos, la granularidad es reducida y el desbalance persiste.

La Opción 1 (paralelo en celdas) contribuye de forma mínima porque se aplica con `inner_threads=1` dentro de cada hilo externo: en la configuración actual, el paralelismo de celdas está efectivamente deshabilitado para evitar sobresubscripción.

5.2. Impacto de las optimizaciones de memoria

La versión Memoria+OpenMP es consistentemente **más lenta** que la versión paralela en ≈ 5–10 %. Este resultado, aparentemente contraintuitivo, se explica por los siguientes factores:

1. **El cuello de botella no es el ancho de banda de memoria**: la operación dominante en cada celda es el cálculo de índices de vecinos con módulo (`left` $= (i{-}1{+}N)\,\%N$, `right` $= (i{+}1)\,\%N$), que es aritmética de enteros pura. Aunque el `uint8_t` reduce la presión de memoria, el tiempo está dominado por esta aritmética.

2. **El overhead del `malloc` es despreciable**: con solo 21 iteraciones de densidad y cada hilo allocando solo dos arreglos por iteración, la contención del asignador descrita en `MemoryExplanation.md` es mínima en la práctica. El pool pre-allocado elimina un overhead ya pequeño.

3. **El acceso con `uint8_t` puede generar instrucciones adicionales**: el compilador debe manejar extensiones de tipo al mezclar aritmética de `int` (índices) con lecturas de `uint8_t` (celdas), compensando la ventaja de la mayor densidad de datos por línea de caché.

4. **La diferencia está dentro del margen de variabilidad**: la diferencia entre paralelo y memoria ($\approx$ 3–3,5 s para $N = 300\,000$) es del mismo orden que la variación entre las dos repeticiones de un mismo binario ($\approx$ 0,4 s), lo que sugiere que el `uint8_t` no perjudica, pero tampoco mejora de forma significativa en estas condiciones.

En definitiva, las optimizaciones de memoria son correctas en principio pero su beneficio se materializaría para N mucho mayor (donde los arreglos excedan la caché L3) o para un número de iteraciones de densidad significativamente más alto.

5.3. Escalado con N y tendencia asintótica

La Figura 2 muestra que el speedup de la versión paralela crece con N, partiendo de 3,41× en $N = 10\,000$ y llegando a 4,12× en $N = 300\,000$. Este comportamiento es característico de la paralelización de bucles externos: para N pequeño, el overhead de creación y sincronización de hilos representa una fracción mayor del trabajo total; para N grande, cada hilo tiene más trabajo genuino que amortigua ese overhead.

La versión Memoria+OpenMP muestra el mismo patrón de escalado, pero con un desplazamiento hacia abajo de $\approx$ 0,1–0,2× de speedup. Esto confirma que las dos implementaciones tienen el mismo comportamiento asintótico y que la diferencia es un factor constante de overhead, no una diferencia algorítmica estructural.

5.4. Comparación entre las tres implementaciones

Implementación	**Tiempo $N = 300k$ (s)**	**Speedup vs Seq**	**Factor vs Paralelo**
Secuencial	287.77	1,00×	4,12× más lento
Paralelo (OpenMP)	69.83	4,12×	1,00× (referencia)
Memoria + OpenMP	73.24	3,93×	1,05× más lento

Cuadro 5: Comparativa de implementaciones para $N = 300\,000$ (promedios de 2 repeticiones).

La Tabla 5 sintetiza el resultado central del reto: el paralelismo con OpenMP produce una mejora real y sustancial (4×) sobre la versión serial. Las optimizaciones de memoria no generan una mejora adicional en las condiciones del benchmark, pero tampoco perjudican significativamente el rendimiento, y mejoran la calidad del código (menor uso de memoria, ausencia de asignaciones dinámicas en la sección crítica).

6. Conclusiones

1. **OpenMP produce un speedup real y sostenido de** 3,4×–4,1×: la estrategia de paralelizar el bucle externo de densidades (Opción 2, *embarrassingly parallel*) aprovecha eficazmente los núcleos del M4. Este speedup crece suavemente con N, confirmando que la paralelización se amortiza mejor para cargas de trabajo mayores.

2. **La solución serial es el punto de partida imprescindible**: implementar primero la versión secuencial correcta permitió identificar con claridad dónde está el trabajo computacional (el barrido de densidades) y qué invariantes debe preservar la versión paralela (orden de barrido independiente, semillas distintas por hilo, sin escrituras compartidas entre hilos).

3. **La estrategia *embarrassingly parallel* (Opción 2) es más efectiva que el paralelismo de grano fino (Opción 1)**: al operar sobre densidades completamente independientes sin ninguna sincronización intermedia, se evita el overhead de barreras y reducciones que introduciría la paralelización del bucle de celdas en cada uno de los $N/5$ pasos de simulación.

4. **Las optimizaciones de memoria no producen mejora medible en estas condiciones**: el cuello de botella del autómata celular es la aritmética de índices (operación módulo), no el acceso a memoria. Para tamaños de carretera $N \leq 300\,000$, los arreglos caben cómodamente en caché incluso con `int`, y la reducción a `uint8_t` no aporta ventaja observable. Sería relevante para N del orden de millones.

5. **La arquitectura heterogénea del M4 limita el speedup a $\approx 4\times$**: los 6 E-cores son considerablemente más lentos que los 4 P-cores. Con 21 iteraciones de densidad distribuidas entre 10 hilos, los hilos en E-cores generan un desbalance de carga que se convierte en el factor limitante, independientemente de la estrategia de paralelización.

6. **El condicional en el Makefile para Apple Silicon es esencial para la portabilidad**: la detección de `Darwin` y el uso de las rutas de Homebrew para `libomp` hace que el proyecto compile y ejecute correctamente tanto en macOS como en Linux sin modificaciones manuales.

7. **El sistema de checkpoints del script de benchmarking es una práctica robusta de ingeniería**: la capacidad de reanudar automáticamente una corrida interrumpida evita pérdidas de tiempo de cómputo y garantiza la reproducibilidad de los resultados sin lanzar el benchmark completo desde cero.

8. **La medición con `CLOCK_MONOTONIC` es imprescindible para programas paralelos**: mide el tiempo de pared real (no la suma de tiempos de CPU de todos los hilos), asegurando que los resultados reflejan genuinamente la ganancia de velocidad y no un artefacto de medición.

7. Bibliografía

Referencias

[1] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface* (versión 5.2). https://www.openmp.org/specifications/

[2] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of AFIPS Spring Joint Computer Conference*, pp. 483–485.

[3] Wolfram, S. (2002). *A New Kind of Science*. Wolfram Media. (Véase también el documento de referencia `trafficmodel.pdf` adjunto al reto.)

[4] Nagel, K., & Schreckenberg, M. (1992). A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12), 2221–2229.

[5] López, A. (2017). Introducción al High Performance Computing. *Encamina Blog*. https://blogs.encamina.com/por-una-nube-sostenible/introduccion-al-high-performance-computing/

[6] Apple Inc. (2024). *Apple M4 chip*. https://www.apple.com/mac-mini/